Estrutura de pastas

```
controle-financeiro/
├── README.md
├── requirements.txt
├── .gitignore            # ignora .venv, .env e data/financeiro.db
├── .env.example          # modelo de variáveis (chave de API, sem valor real)
├── app.py                # página inicial do Streamlit
├── pages/                # cada arquivo vira uma página do app
│   ├── 1_Dashboard.py
│   ├── 2_Lancamentos.py
│   ├── 3_Orcamento_e_Metas.py
│   ├── 4_Assistente.py
│   └── 5_Importar_Dados.py
├── src/
│   ├── database/
│   │   ├── schema.sql      # criação das tabelas
│   │   ├── connection.py   # abre/fecha conexão, cria o banco
│   │   └── repositories.py # funções de leitura/escrita (SQL fica aqui)
│   ├── services/
│   │   ├── fluxo_caixa.py  # saldo, totais por período, por categoria
│   │   ├── orcamento.py    # limite por categoria x gasto real
│   │   ├── metas.py        # progresso das metas e reservas
│   │   └── relatorios.py   # resumos mensais
│   ├── assistant/
│   │   ├── regras.py       # alertas por regras (Fase 6)
│   │   └── ia.py           # assistente com IA (Fase 7, opcional)
│   ├── charts/
│   │   └── graficos.py     # funções que devolvem figuras Plotly
│   └── utils/
```

```
│   ├── dinheiro.py     # centavos <-> "R$ 1.234,56"
│   └── datas.py        # mês de referência, intervalos
├── data/
│   ├── financeiro.db   # SEU banco real (nunca vai para o Git)
│   └── demo.db         # banco fictício (vai para o Git)
├── scripts/
│   ├── gerar_dados_demo.py
│   └── importar_historico.py
├── tests/
└── docs/               # diagrama do banco, prints para o README
```

Modelo do banco de dados

```
CREATE TABLE categorias (
  id     INTEGER PRIMARY KEY,
  nome   TEXT NOT NULL UNIQUE,
  tipo   TEXT NOT NULL CHECK (tipo IN ('entrada','saida')),
  ativa  INTEGER NOT NULL DEFAULT 1
);

CREATE TABLE contas (
  id          INTEGER PRIMARY KEY,
  nome        TEXT NOT NULL UNIQUE,       -- ex.: Carteira, Conta corrente
  tipo        TEXT NOT NULL,              -- carteira, corrente, poupanca, cartao
  saldo_inicial  INTEGER NOT NULL DEFAULT 0    -- em centavos
);

CREATE TABLE transacoes (
  id          INTEGER PRIMARY KEY,
  data        TEXT NOT NULL,              -- 'AAAA-MM-DD'
  valor       INTEGER NOT NULL CHECK (valor > 0),   -- centavos
  tipo        TEXT NOT NULL CHECK (tipo IN ('entrada','saida')),
  categoria_id    INTEGER NOT NULL REFERENCES categorias(id),
  conta_id        INTEGER NOT NULL REFERENCES contas(id),
  descricao       TEXT,
  forma_pagamento  TEXT,                  -- pix, débito, crédito, dinheiro
  criado_em       TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE orcamentos (
```

```
  id          INTEGER PRIMARY KEY,
  categoria_id  INTEGER NOT NULL REFERENCES categorias(id),
  mes         TEXT NOT NULL,              -- 'AAAA-MM'
  limite      INTEGER NOT NULL,           -- centavos
  UNIQUE (categoria_id, mes)
);

CREATE TABLE metas (
  id              INTEGER PRIMARY KEY,
  nome            TEXT NOT NULL,        -- ex.: Reserva de emergência
  valor_alvo      INTEGER NOT NULL,
  prazo           TEXT,
  percentual_entrada  REAL DEFAULT 0      -- % de cada entrada a separar
);

CREATE TABLE aportes_meta (
  id       INTEGER PRIMARY KEY,
  meta_id    INTEGER NOT NULL REFERENCES metas(id),
  data     TEXT NOT NULL,
  valor    INTEGER NOT NULL
);
```

Passo a passo

Fase 0: Preparação do ambiente

- **0.1** Instalar Python 3.12+ e Git; criar conta no GitHub (se ainda não tiver).
- **0.2** Criar a pasta controle-financeiro e abrir no VS Code.
- **0.3** Criar o ambiente virtual e instalar as bibliotecas:
- python -m venv .venv
- .venv\Scripts\activate
- pip install streamlit pandas plotly pytest python-dotenv
- pip freeze > requirements.txt
- **0.4** git init, criar .gitignore (com .venv/, .env, data/financeiro.db) e fazer o primeiro commit.
- **0.5** Criar as pastas da estrutura acima (vazias, com __init__.py dentro de src/ e subpastas).
- **Me peça:** o .gitignore e o esqueleto de pastas. **Pronto quando:** streamlit run app.py abre uma página com "Olá".

Fase 1: Banco de dados

- **1.1** Criar schema.sql com as tabelas da seção 2.
- **1.2** Criar connection.py: abre o SQLite, ativa chaves estrangeiras e cria as tabelas se não existirem.
- **1.3** Criar utils/dinheiro.py (converter reais ↔ centavos e formatar R$ 1.234,56).
- **1.4** Criar repositories.py com as funções básicas: inserir, listar, editar e excluir transações; listar categorias e contas.
- **1.5** Primeiros testes em tests/ (inserir e ler uma transação num banco temporário).
- **Me peça:** connection.py, repositories.py e os testes. **Pronto quando:** pytest passa.

Fase 2: Núcleo de negócio + dados fictícios

- **2.1** Criar scripts/gerar_dados_demo.py: gera data/demo.db com 6 meses de movimentações **inventadas** (entradas irregulares de eventos, gastos fixos e variáveis).
- **2.2** Criar services/fluxo_caixa.py: saldo atual, total de entradas/saídas por mês, gasto por categoria, maior gasto do mês.

- **2.3** Testar os serviços com o banco demo (conferir os números "na mão" em alguns casos).
- **Pronto quando:** você consegue pedir "gasto por categoria de março" em código e o resultado confere.

Fase 3: Tela de lançamentos

- **3.1** Criar o formulário de nova entrada/saída (data, valor, tipo, categoria, conta, forma de pagamento, descrição).
- **3.2** Listar transações em tabela com filtros (mês, categoria, tipo).
- **3.3** Editar e excluir lançamentos (com confirmação).
- **3.4** Validações: valor positivo, data válida, categoria compatível com o tipo.
- **3.5** Cadastro de categorias e contas (para você não depender de editar o banco na mão).
- **Pronto quando:** você lança, edita e apaga movimentações fictícias pela interface sem tocar no código.

Fase 4: Dashboard e gráficos

- **4.1** Cartões de resumo: saldo, entradas e saídas do mês, variação em relação ao mês anterior.
- **4.2** Gráfico de gastos por categoria (pizza ou barras).
- **4.3** Gráfico de entradas × saídas por mês (barras agrupadas).
- **4.4** Gráfico de evolução do saldo (linha).
- **4.5** Filtro de período (mês, trimestre, ano, personalizado).
- **Me peça:** charts/graficos.py e a página 1_Dashboard.py. A customização fina dos gráficos (cores, tipos, o que mostrar) você decide agora, com o dashboard já funcionando.

Fase 5: Orçamento e metas

- **5.1** Definir limite mensal por categoria (orcamentos) e mostrar barra de progresso "gasto × limite".
- **5.2** Criar metas (ex.: reserva de emergência) com valor-alvo e prazo.
- **5.3** Registrar aportes e mostrar o progresso de cada meta.
- **5.4** Regra de reserva automática: sugerir separar X% de cada entrada nova para a meta. Isso ataca direto o seu problema de "não sobra no fim do mês".
- **Pronto quando:** ao lançar uma entrada fictícia, o app mostra quanto deveria ir para a reserva.

Fase 6: Assistente por regras (sem IA)

- **6.1** Criar assistant/regras.py com regras simples e transparentes, por exemplo:
    - gasto da categoria passou de 80% e de 100% do limite;
    - gasto do mês está acima da média dos últimos 3 meses;
    - ritmo de gasto indica que o dinheiro acaba antes do fim do mês;
    - entrada recebida e ainda sem aporte na reserva.
- **6.2** Mostrar os alertas na página "Assistente" e como aviso no dashboard.
- **6.3** Cada regra devolve uma mensagem clara e uma sugestão de ação.
- **Pronto quando:** com o banco demo, aparecem alertas coerentes e você consegue explicar por que cada um disparou.

Fase 7: Assistente com IA (opcional, mas forte no portfólio)

- **7.1** Decidir o formato: **chat** na página "Assistente" que responde perguntas ("onde mais gastei este mês?", "dá para comprar X?") com base nos seus números.
- **7.2** Guardar a chave de API no .env (nunca no código nem no Git).
- **7.3** Montar o contexto enviado à IA com **dados agregados** (totais por categoria, saldo, metas), sem descrições individuais, por privacidade.
- **7.4** Escrever o prompt de sistema (papel do assistente, tom, limites: ele sugere, não decide por você).
- **7.5** Tratar erros (sem internet, chave inválida) e manter as regras da Fase 6 como alternativa.
- **Atenção:** o uso de API de IA é pago por consumo. Para uso pessoal costuma ser barato, mas vale conferir o preço antes. O projeto continua completo sem esta fase.

Fase 8: Acabamento e portfólio

- **8.1** Revisar nomes, remover código morto e rodar todos os testes.
- **8.2** Escrever o README.md: problema que resolve, funcionalidades, stack, como instalar e rodar, prints, decisões técnicas, próximos passos.
- **8.3** Tirar prints **usando apenas demo.db** (nunca seus dados reais).
- **8.4** Diagrama do banco em docs/ (pode ser feito em Mermaid ou dbdiagram).
- **8.5** Publicar no GitHub e conferir que data/financeiro.db e .env **não** subiram.
- **8.6** Destacar no LinkedIn e no portfólio (com os certificados de Python, dados e IA como contexto).

Fase 9: Dados reais (passos finais)

- **9.1 Backup e segurança:** confirmar o .gitignore e criar o hábito de copiar data/financeiro.db para um local seguro (por exemplo, sua pasta do Dropbox).
- **9.2** Criar o banco real vazio (financeiro.db) e configurar o app para usá-lo (o demo.db fica só para demonstração).
- **9.3** Cadastrar suas **contas reais** e o saldo inicial de cada uma.
- **9.4** Ajustar as **categorias** à sua vida real (ex.: Chronomax, estudos, lazer, namorada/presentes, transporte, alimentação, assinaturas).
- **9.5** Cadastrar os **gastos e receitas fixos**.
- **9.6** Importar o histórico: escolher entre lançar o mês atual em diante ou importar extratos antigos (CSV/OFX do banco) com importar_historico.py. Recomendo começar pelo mês atual.
- **9.7** Definir os **primeiros orçamentos** (com base nos seus gastos reais do último mês) e a **meta de reserva** com a porcentagem de cada entrada.
- **9.8** Usar por 2 a 4 semanas lançando tudo, e depois fazer o **primeiro fechamento de mês**: o que o dashboard e o assistente mostraram, o que ajustar nas regras e nos limites.
- **Pronto quando:** você fecha um mês inteiro com dados reais e saldo do app batendo com o saldo do banco.